

В обектно-ориентирания дизайн е важно какво прави класа, как е свързан с други класове, кой кого притежава, колко силна е зависимостта.

Dependency - клас А използва клас В временно
 - без ownership и е краткотрайно
 - най-слабата форма на връзка

• Добри практики: dependency чрез параметри / чрез абстракция
 • Лоша практика е създаването на обекти вътре в метода

Ownership - описва кой обект/променлива е отговорен за живота на даден ресурс и кой трябва да я освободи

Пример: Динамично разпределена памет:

- Ако обект притежава указател, той трябва да се грижи за *delete* или да използва *smart pointer* за автоматично управление на паметта.

Чрез правилното управление на *ownership* се избягва тегогу *leaks*, двойно изтриване

Златно правило:

- Запосва се от най-простото и се добавя сложност само при нужда.
- Ако искаме да имаме член-данна на класа я правим:
 - 1) *Stoynost* - когато обектът е малък/лесен за местене; най-бързия и безопасен ^{кор}
 - 2) *unique_ptr* - обектът е голям, полиморфен или трябва да живее извън живота на класа, има само един собственик
 - 3) *shared_ptr* - обектът има повете от един собственик
 - 4) Референция (T&) - функцията използва обекта за малко и той съществува
 - 5) *Raw Pointer* (T*) - не го притежаваме, друг се грижи за живота му

Композиция - обектът притежава другия обект
 - унищожение на цялото $\Rightarrow$ унищожение и на частта
 - реализира се чрез стойност в класа

Агрегация - обектът не притежава другия
 - всеки има свой жизнен цикъл
 - реализира се чрез указател, референция

Design Guidelines

1. Ясна отговорност
2. Слаба свързаност, силна кохезия
3. Изразителни интерфейси
4. Грешките трябва да са очевидни
5. Композицията е за предпочитане пред наследяването
6. Проектиране за промяна, не за текущия момент

std::variant - type-safe обединение



- позволява една променлива да съдържа точно една стойност от предварително зададен набор типове, като компилаторът следи кой тип е активен.

вътрешно съхранява: 1) памет, достатъчна за най-големия тип
2) индекс (кой тип е активен)

• Само един тип е активен в даден момент.

При смяна → старият обект се унищожава, новият се конструира